

附 录

附 A 项目代码

附 A1.1 任务分解

```
user_input=input("你想设计什么?")
input_message=user_input
condition=True
while condition:
    scheme=designer.generate_reply(messages=[{"role": "user", "content": input_message}])
    print(scheme)
    if "design scheme" in scheme:
        condition=False
    else:
        user_input=input("你想添加什么内容?")
        input_message="你的上一个回答是: \n"+scheme+"\n对此, 用户的添加内容为: \n"+user_input
```

附 A1.2 格式转换

```
client = OpenAI(api_key=API_KEY, base_url=BASE_URL)

prompt=''
你是一个 Verilog 专家助理, 负责将用户的设计需求转换为适合代码生成模型的结构化输入, 你的输出需要让shailja/fine-tuned-codegen-2B-Verilog模型理解。请严格遵循以下规则:

需求分解
1. 提取所有功能需求
2. 提取物理接口: "输入" "输出" "信号"等
3. 标注约束条件:

输出必须严格遵守, 且输出必须为英文, 你的工作只是转换设计需求的格式, 不需要给出具体代码:
//module <模块名称>
module module_name #(
    // input: <用户描述的输入特征>
    input 类型 [位宽] 信号名,
    // output: <用户描述的输出特征>
    output 类型 [位宽] 信号名
);
// functions: <自然语言描述其实现的功能>
// constraints: <关键约束>
...

input=scheme
response = client.chat.completions.create(
    model="deepseek-reasoner",
    messages=[
        {"role": "system", "content": prompt},
        {"role": "user", "content": input},
    ],
    stream=False
)
print(response.choices[0].message.content)

file_path='E:/pycharm/autogen/src/scheme.txt'
open(file_path, 'w', encoding='utf-8-sig').write(response.choices[0].message.content)
```

附 A1.3.v 文件 verilog 生成

```
filepath='E:/pycharm/autogen/src/scheme.txt'
with open(filepath, 'r',encoding='utf-8-sig') as file_object:    prompt=file_object.read()

device = 'cuda' if torch.cuda.is_available() else 'cpu'

# 关键优化后的模型加载
try:
    tokenizer = AutoTokenizer.from_pretrained("shailja/fine-tuned-codegen-2B-Verilog")
    model = AutoModelForCausalLM.from_pretrained(
        pretrained_model_name_or_path: "shailja/fine-tuned-codegen-2B-Verilog",
        device_map="auto",
        load_in_8bit=True,          # 8 位量化
        torch_dtype=torch.float16,  # 半精度
        low_cpu_mem_usage=True      # 低内存模式
    )
except Exception as e:
    print(f"模型加载失败: {e}")
    exit()

input_ids = tokenizer(prompt, return_tensors="pt").input_ids.to(device)
try:
    sample = model.generate(
        input_ids,
        max_length=3000,
        temperature=0.8,
        do_sample=True,
        top_p=0.9,
        pad_token_id=tokenizer.eos_token_id,
    )
    decoded = tokenizer.decode(sample[0], truncate_before_pattern=[r"endmodule"]) + "endmodule"
    print("\n 生成的 Verilog 代码:\n" + decoded)
    file_path="E:/pycharm/autogen/src/programming.v"
    with open(file_path, 'w',encoding='utf-8-sig') as file_object:
        file_object.write(decoded)
except RuntimeError as e:
    print(f"生成失败（显存不足）: {e}")
finally:
    torch.cuda.empty_cache()
```

附 A1.4 testbench 编写

```
verilog_test_writer_system_message1 = """
    You are an expert in writing testbenches using the Verilog-2005 language.

    I will provide you with the user's design requirements and the .v file containing the Verilog code. You are to write the testbench code based on that.

    You must strictly follow the Verilog-2005 syntax specification. This is very important. Do not use any syntax that is not supported in Verilog-2005.

    Your testbench must demonstrate all functionalities of the Verilog code in the .v file, such as how different inputs produce different outputs and how different input scenarios affect the outputs.

    Your response must strictly follow this format:
    It must start with ``timescale 1ns / 1ps" followed by "module programming_testbench;"
    It must end with "endmodule"
    Do not include any words, sentences, or symbols outside of the code.
    Do not begin the code with ```verilog" and do not end it with ````".
    """"

verilog_test_writer=ConversableAgent(
    name="verilog test writer",
    system_message=verilog_test_writer_system_message1,
    llm_config={"config_list": [{"model": "deepseek-reasoner", "api_key": API_KEY,"base_url":BASE_URL}]},
    code_execution_config=False, # Turn off code execution, by default it is off.
    function_map=None, # No registered functions, by default it is None.
    human_input_mode="NEVER", # Never ask for human input.
)

prompt="设计方案为：\n"+need+"\n.v文件的代码为：\n"+code

testbenchcode=verilog_test_writer.generate_reply(messages=[{"content": prompt, "role": "user"}])
print(testbenchcode)

filepath='E:/pycharm/autogen/src/programming_testbench.v'
filename = 'programming_testbench.v'
with open(filepath, 'w') as file_object:
    file_object.write(testbenchcode)
```

附 A1.5 TCL 脚本

```
# 定义项目路径
set project_dir "E:/pycharm/autogen/"
set src_dir "${project_dir}/src"

# 创建新项目
create_project vivado_project ${project_dir}/vivado_project -force

# 添加源文件和测试文件
add_files "${src_dir}/programming.v"
add_files "${src_dir}/programming_testbench.v" -filesset sim_1

# 设置顶层模块
set_property top programming_testbench [get_filesets sim_1]

# 仿真设置
set_property sim_mode behavioral [get_filesets sim_1]
set_property target_simulator xsim [get_filesets sim_1]

# 运行仿真
launch_simulation

# 打开波形窗口（仿真 GUI 环境会自动加载）
start_gui
```

附 A1.6 加载 TCL 脚本

```
def run_vivado(): 2 用法
    import subprocess

    # Vivado 路径和 Tcl 脚本路径
    vivado_path = r"E:/vivado/Vivado/2023.1/bin/vivado.bat"
    tcl_script = r"E:/pycharm/autogen/tcl_script1.tcl"

    try:
        # 运行 Vivado 并加载 Tcl 脚本
        subprocess.run( args: [vivado_path, "-mode", "batch", "-source", tcl_script], shell=True, check=True)
        print("Vivado 仿真、综合和波形显示已完成! ")
        return False
    except subprocess.CalledProcessError as e:
        print(f"运行 Vivado 时发生错误: {e}")
        return True
    except FileNotFoundError:
        print("未找到 Vivado 或 Tcl 脚本, 请检查路径! ")
        return "未找到 Vivado 或 Tcl 脚本, 请检查路径! "
```

附 A1.7 debug

```
API_KEY = os.getenv('MY_API_KEY')
BASE_URL = os.getenv('MY_BASE_URL')

def debug(scheme,verilog_code,testbench):
    file_path = "E:/pycharm/autogen/vivado.log"
    with open(file_path,'a') as f:
        vivado_log = f.read()
    os.remove(file_path)
    input="设计方案为: \n"+scheme+"verilog的代码为: \n"+verilog_code+"\n"+testbench的代码为\n"+testbench+"\n"+vivado的日志: \n"+vivado_log

    debug_agent_system_message="""
1. You are an agent responsible for debugging Verilog code.
2. The user will provide their design scheme, the Verilog '.v' file code, the testbench code, and Vivado software logs. You are to use this information to correct the Verilog code in the '.v' file.
3. Important note: Vivado supports Verilog-2005. Errors in the provided '.v' file are very likely due to syntax that does not comply with the Verilog-2005 standard. Your corrected code must also st
4. Only correct the Verilog code in the '.v' file. Do not modify the testbench - editing the testbench is the responsibility of another agent.
5. Your final answer must strictly start with 'module' and end with 'endmodule'. Do not include any words or sentences outside of the code. Do not start the code with '```verilog` and do not end:
"""

    debug_agent = ConversableAgent(
        name="designer",
        system_message=debug_agent_system_message,
        llm_config={"config_list": [{"model": "deepseek-reasoner", "api_key": API_KEY, "base_url": BASE_URL}]},
        code_execution_config=False, # Turn off code execution, by default it is off.
        function_map=None, # No registered functions, by default it is None.
        human_input_mode="NEVER", # Never ask for human input.
    )

    debug_code=debug_agent.generate_text(
        """
        """
    )
```